MODERN ACADEMY FOR ENGINEERING & TECHNOLOGY

Electronic Eng. and Communication Tech. Branch

DSP Lecture Report – Fall 27 – Law 20

Prepared by:

St. Name	Branch	Section	Level
مصطفى محمد السعيد محمد البكش	حاسبات	5L-TH6	الرابع

Under supervision of:

Dr. / Samir M. Kamal

Cairo 2027

DSP – Lecture Report Instruction – Fall 2027- Law 20

The Report consists of two parts, as shown:

1- **For digital filters, review the following items: types, uses, advantages and limitations, choosing between them and in details, steps of design. (15 to 20 pages).**

2- **Describe Laplace transform, showing its applications and show its Relationship with Z- Transform. (8 to 10 pages).**

- **The report must be written in word by Times New Roman, font 13 and linespace 1.5. Totally not exceed 30 pages.**
- **Each student must upload two copies of the report in word and pdf to the moodle platform.**
- **Cover page of the report, as given to you at the WhatsApp groub.**
- **Report submission: Thursday 22 / 10 /2026, 8PM.**
- **Note: This page is the first page in your report.**
- **Attchement No.1, includes required examples and problems for chapter 5. It will upload to the coordinator later.**
- **These problems will solve hand writing in very good A4 papers and submit to Eng. M. Khalaf in Tuesday 01 / 12 /2026.**

(Samir Kaml)

Dr. Samir M. Kamal

Course Instructor

TABLE OF CONTENTS

PART 1: DIGITAL FILTERS....4
1.1 Introduction....4
1.2 Uses of Digital Filters....4
1.3 Advantages of Digital Filters....5
1.4 Limitations of Digital Filters....6
1.5 Types of Digital Filters: FIR and IIR....6
1.6 Choosing Between FIR and IIR Filters....7
1.7 Steps of Digital Filter Design....10
1.7.1 Specification of the filter requirements....10
1.7.2 Calculation of suitable filter coefficients....12
1.7.3 Representation of the filter by a suitable structure (realization)....13
1.7.4 Analysis of finite word length effects....15
1.7.5 Implementation of the filter....16
1.8 Illustrative Examples....17
PART 2: THE LAPLACE TRANSFORM....20
2.1 Continuous-Time Signals and Systems....20
2.2 Description of the Laplace Transform....21
2.3 Applications of the Laplace Transform....21
2.4 From Continuous Time to Discrete Time: Sampling....22
2.5 The Z-Transform....24
2.6 Relationship Between the Laplace Transform and the Z-Transform....26
2.7 Numerical Illustration of the Mapping....28
2.8 Summary of the Relationship....29
References....29

PART 1: DIGITAL FILTERS

1.1 Introduction

A filter is a system or network that selectively changes the wave shape, the amplitude–frequency characteristic and/or the phase–frequency characteristic of a signal in a desired manner. The common filtering objectives are to improve the quality of a signal (for example, to remove or reduce noise), to extract information from signals, or to separate two or more signals that were previously combined, for example to make efficient use of an available communication channel.

A digital filter is a mathematical algorithm, implemented in hardware and/or software, that operates on a digital input signal to produce a digital output signal in order to achieve a filtering objective. The term digital filter refers to the specific hardware or software routine that performs the filtering algorithm. Digital filters often operate on digitized analogue signals, or just on numbers, representing some variable, that are stored in a computer memory.

Fig. 1 shows a simplified block diagram of a real-time digital filter with analogue input and output signals. The band-limited analogue signal is sampled periodically and converted into a series of digital samples x(n), n = 0, 1, …. The digital processor implements the filtering operation by mapping the input sequence x(n) into the output sequence y(n) in accordance with the computational algorithm of the filter. The digital-to-analogue converter (DAC) converts the digitally filtered output into analogue values, which are then analogue filtered to smooth them and to remove unwanted high-frequency components.

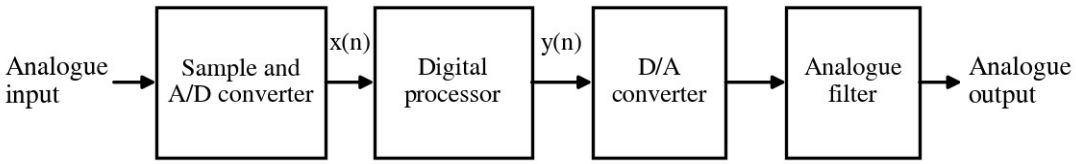


Fig. 1 A simplified block diagram of a real-time digital filter with analogue input and output signals.

1.2 Uses of Digital Filters

The filtering objectives given in Section 1.1 (noise reduction, extraction of information and separation of combined signals) define what filters are used for. Digital filters play very important roles in digital signal processing, and compared with analogue filters they are preferred in a number of applications, because of the advantages listed in Section 1.3. The applications mentioned in the course reference are:

- Data compression;

- Biomedical signal processing;
- Speech processing;
- Image processing;
- Data transmission;
- Digital audio;
- Telephone echo cancellation.

Several further uses follow from the nature of digital filters. Because the frequency response of a digital filter can be adjusted automatically when it is implemented on a programmable processor, digital filters are widely used as adaptive filters. Digital filters can be used at very low frequencies, which are found in many biomedical applications and where analogue filters are impractical. In addition, filters are sometimes used to equalize or compensate the phase response of a system, or as phase shifters (Section 1.7.1). A practical use is given in Example 5.3 (Section 1.8), where a digital filter is designed for real-time physiological noise reduction in a biomedical signal processing project.

1.3 Advantages of Digital Filters

Compared with analogue filters, digital filters have one or more of the following advantages:

1. **Characteristics not possible with analogue filters.** A digital filter can have, for example, a truly linear phase response.
2. **Stable performance.** The performance of a digital filter does not vary with environmental changes such as thermal variations, which eliminates the need to calibrate the filter periodically.
3. **Programmability.** When a programmable processor is used, the frequency response can be adjusted automatically, which is why digital filters are widely used as adaptive filters.
4. **Shared hardware.** Several input signals or channels can be filtered by one digital filter without the need to replicate the hardware.
5. **Data storage.** Both filtered and unfiltered data can be saved for further use.
6. **VLSI technology.** The advances in VLSI technology can be used to fabricate digital filters that are small in size, consume low power and keep the cost down.
7. **High precision.** The precision achievable with analogue filters is restricted; typically only about 60 to 70 dB of stopband attenuation is possible with active filters designed with off-the-shelf components. With digital filters the precision is limited only by the word length used.
8. **Repeatability.** The performance of digital filters is repeatable from unit to unit.

9. **Very low frequencies and wide frequency range.** Digital filters can be used at very low frequencies, found in many biomedical applications, where analogue filters are impractical. They can also be made to work over a wide range of frequencies by a mere change of the sampling frequency.

1.4 Limitations of Digital Filters

The main disadvantages of digital filters compared with analogue filters are the following:

1. **Speed limitation.** The maximum bandwidth of the signals that digital filters can handle in real time is much lower than for analogue filters. The conversion time of the ADC and the settling time of the DAC limit the highest frequency that can be processed. In addition, the speed of operation depends on the speed of the digital processor and on the number of arithmetic operations required by the filtering algorithm, and this number increases as the filter response is made tighter.
2. **Finite word length effects.** Digital filters are subject to ADC noise resulting from quantizing a continuous signal, and to round-off noise incurred during computation. With higher-order recursive filters, the accumulation of round-off noise could lead to instability.
3. **Long design and development times.** The design and development times, especially for hardware, can be much longer than for analogue filters. However, once developed, the hardware and/or software can be used for other filtering or DSP tasks with little or no modification. Good computer-aided design (CAD) support can make the design of digital filters an enjoyable task, but some expertise is required to use such design aids effectively.

1.5 Types of Digital Filters: FIR and IIR

Digital filters are broadly divided into two classes: infinite impulse response (IIR) filters and finite impulse response (FIR) filters. In its basic form, either type can be represented by its impulse response sequence h(k), k = 0, 1, …. The input and output signals of the filter are related by the convolution sum, given in Eq. (5.1) for the IIR filter and in Eq. (5.2) for the FIR filter.

$$y(n) = \sum_{k=0}^{\infty} h(k)\, x(n-k) \tag{5.1}$$

$$y(n) = \sum_{k=0}^{N-1} h(k)\, x(n-k) \tag{5.2}$$

It is evident from these equations that the impulse response of an IIR filter is of infinite duration, whereas that of an FIR filter is of finite duration, since h(k) has only N values. In practice it is not feasible to compute the output of an IIR filter with Eq. (5.1), because

the impulse response is too long (infinite in theory). Instead, the IIR filtering equation is expressed in a recursive form, Eq. (5.3), where a_k and b_k are the coefficients of the filter.

$$y(n) = \sum_{k=0}^{N} h(k)\,x(n-k) = \sum_{k=0}^{N} a_k\,x(n-k) - \sum_{k=1}^{M} b_k\,y(n-k) \tag{5.3}$$

Equations (5.2) and (5.3) are therefore the difference equations of the FIR and IIR filters respectively. These equations, and in particular the values of h(k) for an FIR filter or of a_k and b_k for an IIR filter, are often the main objectives of a filter design problem. In Eq. (5.3) the current output sample y(n) is a function of past outputs as well as present and past input samples; in other words, the IIR filter is a feedback system of some sort. In the FIR equation, by contrast, y(n) depends only on the present and past values of the input. When all b_k are set to zero, Eq. (5.3) reduces to the FIR Eq. (5.2).

Alternative representations of the FIR and IIR filters are their transfer functions, Eqs. (5.4a) and (5.4b), which are very useful in evaluating the frequency responses of the filters.

$$H(z) = \sum_{k=0}^{N-1} h(k)\,z^{-k} \tag{5.4a}$$

$$H(z) = \frac{\sum_{k=0}^{N} a_k\,z^{-k}}{1 + \sum_{k=1}^{M} b_k\,z^{-k}} \tag{5.4b}$$

The factors that influence the choices open to the designer at each stage of the design process are strongly linked to whether the filter is IIR or FIR. It is therefore very important to appreciate the differences between the two types, their peculiar characteristics and, more importantly, how to choose between them.

1.6 Choosing Between FIR and IIR Filters

The choice between FIR and IIR filters depends largely on the relative advantages of the two filter types:

1. **Phase response.** FIR filters can have an exactly linear phase response, which means that no phase distortion is introduced into the signal by the filter. This is an important requirement in many applications, for example data transmission, biomedicine, digital audio and image processing. The phase responses of IIR filters are nonlinear, especially at the band edges.
2. **Stability.** FIR filters realized non-recursively, that is by direct evaluation of Eq. (5.2), are always stable. The stability of IIR filters cannot always be guaranteed.

3. **Finite word length.** The effects of using a limited number of bits, such as round-off noise and coefficient quantization errors, are much less severe in FIR filters than in IIR filters.
4. **Number of coefficients.** FIR filters require more coefficients than IIR filters for sharp cutoff responses. For a given amplitude response specification, more processing time and storage are therefore required for an FIR implementation. However, one can readily take advantage of the computational speed of the FFT and of multi-rate techniques to improve the efficiency of FIR implementations significantly.
5. **Analogue counterpart.** Analogue filters can be readily transformed into equivalent IIR digital filters meeting similar specifications. This is not possible with FIR filters, as they have no analogue counterpart. However, it is easier to synthesize FIR filters of arbitrary frequency responses.
6. **Ease of synthesis.** In general, an FIR filter is algebraically more difficult to synthesize if CAD support is not available.

From these points a broad guideline on when to use each type can be drawn:

- Use an **IIR** filter when the only important requirements are a sharp cutoff and high throughput, because IIR filters, especially those using elliptic characteristics, give fewer coefficients than FIR filters.
- Use an **FIR** filter if the number of filter coefficients is not too large and, in particular, if little or no phase distortion is desired. Newer DSP processors also have architectures tailored to FIR filtering, and some are designed specifically for FIR filters.

Example 5.1: comparing an IIR and an FIR filter

Two filters meet identical amplitude–frequency response specifications. Filter (1) has the transfer function below, with $a_0 = 0.4981819$, $a_1 = 0.9274777$, $a_2 = 0.4981819$, $b_1 = -0.6744878$ and $b_2 = -0.3633482$.

$$H(z) = \frac{a_0 + a_1 z^{-1} + a_2 z^{-2}}{1 + b_1 z^{-1} + b_2 z^{-2}}$$

Filter (2) has the transfer function below, with $h(0) = 0.5460328 \times 10^{-2} = h(11)$, $h(1) = -0.4506875 \times 10^{-1} = h(10)$, $h(2) = 0.6916942 \times 10^{-1} = h(9)$, $h(3) = -0.5538437 \times 10^{-1} = h(8)$, $h(4) = -0.6342841 \times 10^{-1} = h(7)$ and $h(5) = 0.578924 \times 10^{0} = h(6)$.

$$H(z) = \sum_{k=0}^{11} h(k) z^{-k}$$

(a) Type of each filter. Filter (1) is an IIR filter, because its denominator contains feedback coefficients. Filter (2) is an FIR filter, because it has a finite number of coefficients (12 values of h(k)).

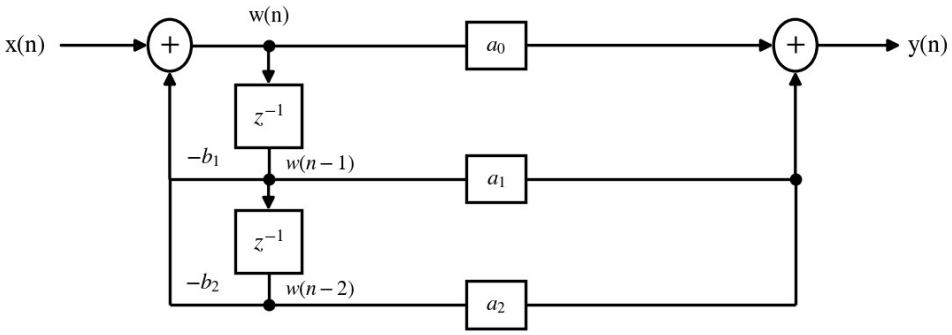
(b) Block diagrams and difference equations. The block diagrams are shown in Fig. 2. The difference equations of filter (1) are given below, and the difference equation of filter (2) is a convolution sum over its 12 coefficients.

$$w(n) = x(n) - b_1 w(n-1) - b_2 w(n-2)$$

$$y(n) = a_0 w(n) + a_1 w(n-1) + a_2 w(n-2)$$

$$y(n) = \sum_{k=0}^{11} h(k)\, x(n-k)$$

(a) IIR filter (second order)



(b) FIR filter, transversal structure (12 coefficients)

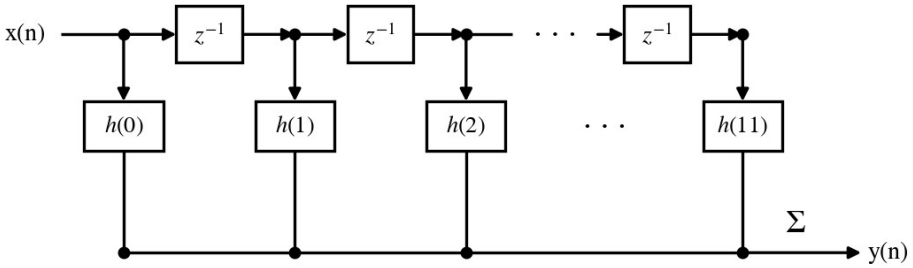


Fig. 2 Block diagram representation of (a) the IIR filter and (b) the FIR filter of Example 5.1.

(c) Computational and storage requirements. Examination of the two difference equations gives the requirements in Table 1.

Table 1 Computational and storage requirements of the filters of Example 5.1

Requirement	FIR	IIR
Number of multiplications	12	5
Number of additions	11	4
Storage locations (coefficients and data)	24	8

The IIR filter is more economical in both computation and storage than the FIR filter. For the same amplitude response specifications, the number of FIR coefficients (12 in this example) is typically six times the order of the IIR transfer function (2 in this case), the order being the highest power of z in the denominator.

1.7 Steps of Digital Filter Design

The design of a digital filter involves five steps:

1. Specification of the filter requirements;
2. Calculation of suitable filter coefficients;
3. Representation of the filter by a suitable structure (realization);
4. Analysis of the effects of finite word length on filter performance;
5. Implementation of the filter in software and/or hardware.

The five steps are not necessarily independent, nor always performed in the order given; to arrive at an efficient filter it may be necessary to iterate a few times between them, especially if the problem specification is not watertight.

1.7.1 Specification of the filter requirements

The requirement specification includes the following:

1. **Signal characteristics:** the types of signal source and sink, the I/O interface, the data rates and width, and the highest frequency of interest.
2. **Filter characteristics:** the desired amplitude and/or phase responses and their tolerances (if any), the speed of operation and the mode of filtering (real time or batch).
3. **Manner of implementation:** for example, as a high-level-language routine in a computer or as a DSP-processor-based system, and the choice of signal processor.
4. **Other design constraints:** for example, the cost of the filter.

The designer may not have enough information to specify the filter completely at the outset, but as many of the requirements as possible should be specified in order to simplify the design process.

The characteristics of digital filters are often specified in the frequency domain. For frequency-selective filters, such as lowpass and bandpass filters, the specifications are usually given as a tolerance scheme. Fig. 3 depicts such a scheme for a lowpass filter. The horizontal lines indicate the tolerance limits: in the passband the magnitude response has a peak deviation of δ_p, and in the stopband it has a maximum deviation of δ_s.

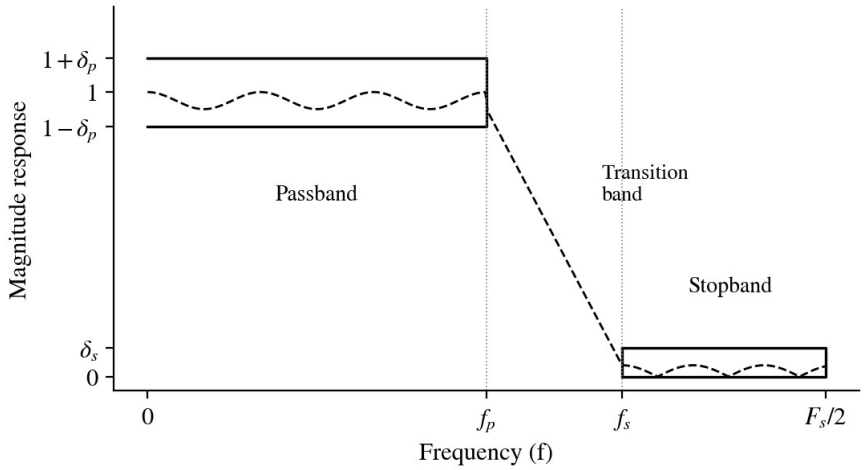


Fig. 3 Tolerance scheme for a lowpass filter.

The width of the transition band determines how sharp the filter is; in this region the magnitude response decreases monotonically from the passband to the stopband. The key parameters of interest are:

- δ_p: the passband deviation;
- δ_s: the stopband deviation;
- f_p: the passband edge frequency;
- f_s: the stopband edge frequency.

The edge frequencies are often given in normalized form, that is as a fraction of the sampling frequency (f/F_s), but specifications in standard units such as hertz or kilohertz are also valid and are sometimes more meaningful, especially to an inexperienced designer. The passband and stopband deviations may be expressed as ordinary numbers or in decibels, in which case they give the passband ripple and the minimum stopband attenuation respectively. For FIR filters, the minimum stopband attenuation A_s and the peak passband ripple A_p, in decibels, are given by Eqs. (5.5a) and (5.5b).

$$A_s = -20\log_{10}\delta_s \tag{5.5a}$$

$$A_p = 20\log_{10}(1+\delta_p) \tag{5.5b}$$

The phase response of a digital filter is often not specified as meticulously as the amplitude response. In many cases it is sufficient to indicate that phase distortion is of concern or that a linear phase response is desirable. However, when filters are used to equalize or compensate the phase response of a system, or as phase shifters, the desired phase response must be specified.

Example 5.2: specification of an FIR bandpass filter

An FIR bandpass filter is to meet the following specifications: passband 0.18–0.33 (normalized), transition width 0.04 (normalized), stopband deviation 0.001 and passband deviation 0.05. Assuming a sampling frequency of 10 kHz, the specifications

in standard units are given in Table 2, and the deviations in decibels follow from Eqs. (5.5a) and (5.5b).

Table 2 Specifications of Example 5.2 in standard units

Quantity	Value
Passband	1.8 – 3.3 kHz
Stopbands	0 – 1.4 kHz and 3.7 – 5 kHz
Stopband attenuation	$-20 \log_{10}(0.001) = 60$ dB
Passband ripple	$20 \log_{10}(1 + 0.05) = 0.42$ dB

1.7.2 Calculation of suitable filter coefficients

In this step one of a number of approximation methods is selected and the coefficients are calculated (h(k) for an FIR filter, or a_k and b_k for an IIR filter) so that the filter characteristics specified in the first step are satisfied. The method depends on whether the filter is of the IIR or the FIR type.

IIR filters. The coefficients of IIR filters are traditionally calculated by transforming known analogue filter characteristics into equivalent digital filters. The two basic methods are the impulse invariant method and the bilinear transformation method. With the impulse invariant method, the impulse response of the original analogue filter is preserved after digitizing it, but its magnitude–frequency response is not. Because of inherent aliasing, the method is inappropriate for highpass or bandstop filters. The bilinear method, on the other hand, yields very efficient filters and is well suited to calculating the coefficients of frequency-selective filters. It allows the design of digital filters with known classical characteristics such as Butterworth, Chebyshev and elliptic. Filters obtained with the bilinear method in general preserve the magnitude response characteristics of the analogue filter but not its time-domain properties. Efficient computer programs now exist that calculate the coefficients with the bilinear method from the filter parameters of interest. The impulse invariant method is good for simulating analogue systems, whereas the bilinear method is best for frequency-selective IIR filters.

The pole-zero placement method is an alternative approach for IIR filters. It is an easy way of calculating the coefficients of very simple filters. For filters with a good amplitude response it is not recommended, because it relies on trial-and-error shuffling of the pole and zero positions.

FIR filters. There are also several methods of calculating FIR coefficients; the reference discusses three: the window method, the frequency sampling method and the optimal method (the Parks–McClellan algorithm). The window method offers a very simple and flexible way of computing the coefficients, but it does not give the designer

adequate control over the filter parameters. The main attraction of the frequency sampling method is that it allows a recursive realization of FIR filters, which can be computationally very efficient; however, it lacks flexibility. The optimal method is now widely used in industry and, for most applications, yields the desired FIR filter. For FIR filters it should therefore be the method of first choice, unless the particular application dictates otherwise or a CAD facility is unavailable. Table 3 summarizes the six methods.

Table 3 Methods of calculating filter coefficients

Type	Method	Main remarks
IIR	Impulse invariant	Preserves the impulse response but not the magnitude response; inherent aliasing; inappropriate for highpass or bandstop filters; good for simulating analogue systems.
IIR	Bilinear transformation	Very efficient; well suited to frequency-selective filters; allows classical Butterworth, Chebyshev and elliptic characteristics; preserves the magnitude response but not the time-domain properties.
IIR	Pole-zero placement	Easy for very simple filters; relies on trial and error; not recommended for filters with a good amplitude response.
FIR	Window	Very simple and flexible; does not give adequate control over the filter parameters.
FIR	Frequency sampling	Allows a recursive realization that can be computationally very efficient; lacks flexibility.
FIR	Optimal (Parks–McClellan)	Widely used in industry; yields the desired FIR filter for most applications; first choice unless the application dictates otherwise or CAD is unavailable.

The method is chosen to suit the particular application, and the choice is influenced mainly by the critical requirements in the specification. In general, the crucial choice is really between FIR and IIR. In most cases, if the FIR properties are vital then the optimal method is a good candidate, whereas if IIR properties are desirable then the bilinear method will in most cases suffice.

1.7.3 Representation of the filter by a suitable structure (realization)

Realization means converting a given transfer function H(z) into a suitable filter structure. Block or flow diagrams are often used to depict filter structures, and they

show the computational procedure for implementing the filter. The structure used depends on whether the filter is IIR or FIR.

IIR structures. Three structures are commonly used: the direct form, the cascade form and the parallel form (Fig. 4). The direct form is simply a straightforward representation of the IIR transfer function. In the cascade form, the transfer function of Eq. (5.4b) is factored and expressed as the product of second-order sections. In the parallel form, H(z) is expanded using partial fractions as the sum of second-order sections. The parallel and cascade structures are the most widely used for IIR filters, because they lead to simpler filtering algorithms and are far less sensitive to the effects of implementing the filter with a finite number of bits than the direct structure. The direct structure suffers from severe coefficient sensitivity problems, especially for high-order filters, and should be avoided in these cases.

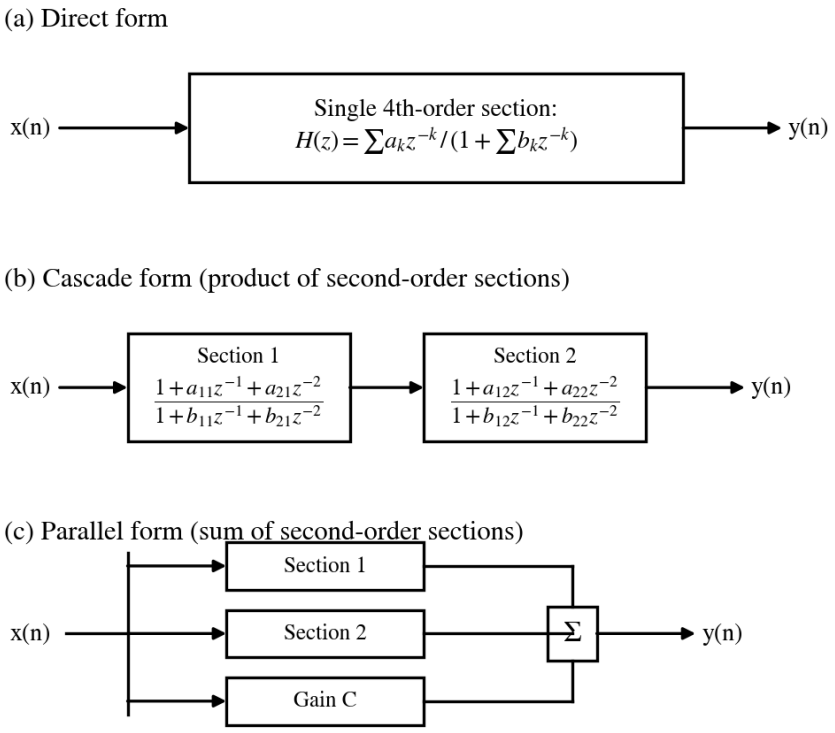


Fig. 4 Realization structures of a fourth-order IIR filter: (a) direct, (b) cascade and (c) parallel.

FIR structures. The most widely used FIR structure is the direct form, because it is particularly simple to implement. It is sometimes called a tapped delay line or transversal filter, because it resembles a tapped delay line (Fig. 2(b)). Two other FIR structures are also used: the frequency sampling structure and the fast convolution technique. Compared with the transversal structure, the frequency sampling structure can be computationally more efficient because it leads to fewer coefficients, but it may not be as simple to implement and requires more storage. Fast convolution uses the

computational advantage of the fast Fourier transform (FFT) and is particularly attractive in situations where the power spectrum of the signal is also required.

Lattice structure. There are many other practical structures, but most are popular only in specific application areas. An example is the lattice structure, which finds use in speech processing and linear prediction applications. It can represent FIR as well as IIR filters. The basic lattice structure is characterized by a single input and a pair of outputs; from it, N-stage FIR lattice structures and, for example, a 2-stage all-pole IIR lattice structure (one with only denominator coefficients) are derived.

For a given filter, the choice between structures depends largely on (i) whether it is FIR or IIR, (ii) the ease of implementation and (iii) how sensitive the structure is to the effects of finite word length.

1.7.4 Analysis of finite word length effects

The approximation and realization steps assume infinite or very high precision. In an actual implementation, however, it is often necessary to represent the filter coefficients with a limited number of bits, typically 8 to 16 bits, and the arithmetic operations of the difference equation are performed using finite-precision arithmetic. The effect is to degrade the performance of the filter and, in some cases, to make it unusable. The designer must therefore analyze these effects and choose suitable word lengths for the filter coefficients, for the filter variables (the input and output samples) and for the arithmetic operations within the filter. The main sources of performance degradation are:

1. **Input/output signal quantization.** In particular, the ADC noise due to quantizing the input signal samples is significant.
2. **Coefficient quantization.** This leads to deviations in the frequency response of both FIR and IIR filters, and possibly to instability in IIR filters.
3. **Arithmetic round-off errors.** Finite-precision arithmetic yields results that need additional bits. When these are quantized to the permissible word length, often by rounding, round-off noise results, which can cause undesirable effects such as instability in an IIR filter.
4. **Overflow.** This occurs when the result of an addition exceeds the permissible word length. It leads to wrong output samples and to possible instability in IIR filters.

The extent of the degradation depends on (i) the word length and the type of arithmetic used for the filtering operation, (ii) the method used to quantize the coefficients and variables to the chosen word length and (iii) the filter structure. From these factors the designer can assess the effects on the filter performance and take remedial action if necessary. Depending on how the filter is implemented, some of the effects may be insignificant: when a filter is implemented as a high-level-language program on most

large computers, coefficient quantization and round-off errors are not important. For real-time processing, finite word lengths (typically 8, 12 and 16 bits) are used to represent the input and output signals, the filter coefficients and the results of the arithmetic operations, and it is then nearly always necessary to analyze the effects of quantization on the filter performance.

1.7.5 Implementation of the filter

After the coefficients have been calculated, a suitable structure chosen, and the degradation caused by quantizing the coefficients and variables verified as acceptable, the difference equation must be implemented as a software routine or in hardware. Whatever the method, the output of the filter must be computed for each sample in accordance with the difference equation (assuming a time-domain implementation). The computation of y(n) involves only multiplications, additions/subtractions and delays, so the following basic building blocks are needed:

- Memory (for example ROM) for storing the filter coefficients;
- Memory (such as RAM) for storing the present and past inputs and outputs, {x(n), x(n − 1), …} and {y(n), y(n − 1), …};
- Hardware or software multiplier(s);
- An adder or arithmetic logic unit.

The designer provides these blocks and ensures that they are suitably configured for the application. The configuration depends largely on whether batch (non-real-time) or real-time processing is required. In **batch processing** the entire data is already available in a memory device, as when experimental data is acquired elsewhere for later analysis. The filter is then often implemented in a high-level language on a general-purpose computer, where all the basic blocks are already configured, so batch processing is essentially a software implementation.

In **real-time processing** the filter is required either (i) to operate on the present input sample x(n) to produce the current output sample y(n) before the next input sample arrives, that is within the inter-sample period, or (ii) to operate on an input block of data, using an FFT algorithm for example, to produce an output block of data within a period proportional to the block length. Real-time filtering may require fast and dedicated hardware if the sample rate is very high or the filter is of high order. For most audio-frequency work, DSP processors such as the DSP56000 (Motorola) and the TMS320C25 (Texas Instruments) are adequate and offer considerable flexibility, since they have all the basic blocks on board, including built-in hardware multiplier(s). In some applications, standard 8-bit or 16-bit microprocessors such as the Motorola 6800 or 68000 families offer attractive alternatives. In addition to the signal processing hardware, the designer must provide suitable input-output interfaces (for example, analogue-to-digital conversion) according to the type of data source and sink.

1.8 Illustrative Examples

Example 5.3: the five design steps applied to a biomedical filter

A digital filter is required for real-time physiological noise reduction with the following amplitude response specifications: passband 0–10 Hz, stopband 20–64 Hz, sampling frequency 128 Hz, maximum passband deviation below 0.036 dB and stopband attenuation above 30 dB. Other important requirements are that (1) minimal distortion of the harmonic relationships between the in-band signal components is highly desirable, (2) the time available for filtering is limited because the filter is part of a larger process, and (3) the filter is implemented with the Texas Instruments TMS32010 DSP processor with the analogue input digitized to 12 bits. The five steps are applied as follows (Table 4).

Table 4 Application of the five design steps to Example 5.3

Design step	Decision and reasoning
1. Requirement specification	The role and performance requirements of the filter, together with the important constraints, are those stated in the problem.
2. Coefficients	Minimal distortion and limited processing time are best achieved with a linear phase FIR filter, with the coefficients obtained using the optimal method.
3. Structure	The transversal structure leads to the most efficient implementation using the TMS32010 processor.
4. Finite word length	Fixed-point arithmetic is used with each coefficient represented by 16 bits (after rounding). The analysis showed that the input quantization noise and the frequency response deviation due to coefficient quantization are both insignificant. Using the 32-bit accumulator to sum the coefficient-data products and rounding only the final sum reduces round-off errors to negligible levels. To avoid overflow, each coefficient is divided by the sum of $\|h(k)\|$ before quantizing to 16 bits.
5. Implementation	Design and configure the TMS32010-based hardware (if it does not already exist) with the necessary input/output interfaces, then write a TMS32010 program that handles the I/O protocols and calculates y(n) for each new input x(n).

Example 5.4: converting an analogue filter into an IIR digital filter

An analogue filter with the transfer function H(s) below is to be converted into an equivalent digital filter operating at a sampling frequency of 256 Hz. The coefficients are to be obtained, and the filter is to be realized in cascade and in parallel structures.

$$H(s)=\frac{1}{s^3+2s^2+2s+1}$$

1. Coefficients. To preserve the amplitude response of the analogue function, the bilinear method is used. Applying it to H(s) gives the digital transfer function:

$$H(z)=\frac{0.1432(1+3z^{-1}+3z^{-2}+z^{-3})}{1-0.1801z^{-1}+0.3419z^{-2}-0.0165z^{-3}}$$

2. Cascade realization. H(z) is factorized into second-order and first-order sections:

$$H(z)=0.1432\frac{1+2z^{-1}+z^{-2}}{1-0.1307z^{-1}+0.3355z^{-2}}\cdot\frac{1+z^{-1}}{1-0.0490z^{-1}}$$

The block diagram follows the cascade form of Fig. 4(b), and the corresponding difference equations are:

$$w_1(n)=0.1432\,x(n)+0.1307\,w_1(n-1)-0.3355\,w_1(n-2)$$
$$y_1(n)=w_1(n)+2\,w_1(n-1)+w_1(n-2)$$
$$w_2(n)=y_1(n)+0.049\,w_2(n-1)$$
$$y_2(n)=w_2(n)+w_2(n-1)$$

3. Parallel realization. H(z) is expressed using partial fractions:

$$H(z)=\frac{1.2916-0.08407z^{-1}}{1-0.131z^{-1}+0.3355z^{-2}}+\frac{7.5268}{1-0.049z^{-1}}-8.6753$$

The block diagram follows the parallel form of Fig. 4(c), and the corresponding difference equations are:

$$w_1(n)=x(n)+0.131\,w_1(n-1)-0.3355\,w_1(n-2)$$
$$y_1(n)=1.2916\,w_1(n)-0.08407\,w_1(n-1)$$
$$w_2(n)=x(n)+0.049\,w_2(n-1)$$
$$y_2(n)=7.5268\,w_2(n)$$
$$y_3(n)=-8.6753\,x(n)$$
$$y(n)=y_1(n)+y_2(n)+y_3(n)$$

Example 5.5: transversal and lattice structures of an FIR filter

The transfer function of an FIR filter is given below. The realization block diagram is required for (1) the transversal structure and (2) a two-stage lattice structure, together with the lattice coefficients.

$$H(z)=1-1.3435z^{-1}+0.9025z^{-2}$$

1. Transversal structure. The diagram follows directly from the transfer function, as in Fig. 2(b) with three coefficients, and the input and output are related by Eq. (5.6).

$$y(n) = x(n) + h(1)\,x(n-1) + h(2)\,x(n-2) \tag{5.6}$$

2. Two-stage lattice structure. The outputs of the lattice structure are related to the input by Eqs. (5.7a) and (5.7b).

$$\begin{aligned} y_2(n) &= y_1(n) + K_2\,w_1(n-1) = x(n) + K_1(1+K_2)\,x(n-1) + K_2\,x(n-2) \\ w_2(n) &= K_2\,x(n) + K_1(1+K_2)\,x(n-1) + x(n-2) \end{aligned} \tag{5.7}$$

Comparing Eq. (5.6) with Eq. (5.7a) and equating coefficients gives the lattice coefficients $K_1 = h(1)/(1 + h(2))$ and $K_2 = h(2)$, which are evaluated below.

$$K_2 = 0.9025, \qquad K_1 = \frac{-1.3435}{1+0.9025} = -0.7062$$

PART 2: THE LAPLACE TRANSFORM

This part presents the Laplace transform as it appears in the course material: the course textbook and the lecture notes of the DSP course. In this material the Laplace transform is the tool that describes continuous-time signals and systems, and the Z-transform is its discrete-time counterpart. The part therefore recalls first the continuous-time signals and systems that the Laplace transform describes, then describes the transform and its applications, then follows the passage from continuous time to discrete time through sampling, then presents the Z-transform, and finally shows the relationship between the two transforms.

2.1 Continuous-Time Signals and Systems

Most signals of practical interest, such as speech, biological, seismic, radar and sonar signals, and communication signals such as audio and video, are analogue signals. An analogue or continuous-time signal is defined for every value of the continuous time variable t, and the subscript a in $x_a(t)$ denotes an analogue signal. The basic example is the continuous-time sinusoidal signal of Eq. (2.1), which is completely characterized by three parameters: the amplitude A, the frequency ω in radians per second (rad/s) and the phase θ in radians.

$$x_a(t) = A\cos(\omega t + \theta), \qquad -\infty < t < \infty \tag{2.1}$$

Instead of ω, the frequency f in cycles per second or hertz (Hz) is often used, where ω and f are related by Eq. (2.2). In terms of f the signal is written $x_a(t) = A\cos(2\pi ft + \theta)$. The sinusoid has the following properties:

$$\omega = 2\pi f \tag{2.2}$$

- For every fixed value of f the signal is periodic, $x_a(t + T_p) = x_a(t)$, where $T_p = 1/f$ is the fundamental period. For f = 0 the period T_p is infinite, which is consistent with the relation $f = 1/T_p$.
- Continuous-time sinusoidal signals with different frequencies are themselves distinct.
- Increasing the frequency f increases the rate of oscillation of the signal, in the sense that more periods are included in a given time interval. Because the time variable t is continuous, the frequency can be increased without limit, so $-\infty < f < \infty$ and $-\infty < \omega < \infty$.

Sinusoidal signals and complex exponentials play a major role in the analysis of signals and systems. In continuous time the basic harmonically related complex exponentials are given by Eq. (2.3).

$$s_k(t) = e^{jk\omega_0 t} = e^{j2\pi k f_0 t}, \qquad k = 0, \pm 1, \pm 2, \ldots \tag{2.3}$$

For each value of k, $s_k(t)$ is periodic with fundamental period $1/(kf_0) = T_p/k$, that is with fundamental frequency kf_0. All the $s_k(t)$ have the common period T_p, the frequency f_0 may take any value, and all members of the set are distinct. A linear combination of these signals, Eq. (2.4), is periodic with fundamental period $T_p = 1/f_0$. Its representation is called the Fourier series expansion of $x_a(t)$; the complex constants c_k are the Fourier series coefficients and the signal $s_k(t)$ is called the k-th harmonic of $x_a(t)$.

$$x_a(t) = \sum_{k=-\infty}^{\infty} c_k s_k(t) = \sum_{k=-\infty}^{\infty} c_k e^{jk\omega_0 t} \tag{2.4}$$

A continuous-time system, like a discrete-time one, is causal if it produces an output only when there is an input, and all physical systems are causal. The Laplace transform, described next, is the transform normally used for such continuous-time systems and signals.

2.2 Description of the Laplace Transform

Continuous-time systems or signals are normally described using the Laplace transform. Its variable is the complex Laplace variable s, given by Eq. (2.5), where d is the real part and ω is the angular frequency.

$$s = d + j\omega \tag{2.5}$$

The values of s are displayed in the s-plane, whose horizontal axis is d and whose vertical axis is jω (Fig. 5). In terms of frequency response the jω axis is the most important axis of the s-plane, since on it d = 0 and its points represent the frequencies of the system.

A continuous-time system is described in the Laplace domain by its transfer function H(s). The analogue low-pass filter of Example 5.4 is of this kind: its transfer function is $H(s) = 1/(s^3 + 2s^2 + 2s + 1)$, a ratio of polynomials in s. Classical analogue filter characteristics such as Butterworth, Chebyshev and elliptic are the kind of continuous-time characteristics that are carried over into digital filters (Section 1.7.2).

The z-transform is the corresponding tool for discrete-time signals and systems. It is a convenient yet invaluable tool for representing discrete-time signals and designing discrete-time systems, and it plays a similar role in discrete-time systems to that which the Laplace transform plays in continuous-time systems. A discrete-time system is essentially a mathematical algorithm that takes an input sequence x(n) and produces an output sequence y(n); examples are digital filters, digital controllers and digital spectrum analysers.

2.3 Applications of the Laplace Transform

Within the course material, the Laplace transform is used in the following ways (Table 5).

Table 5 Applications of the Laplace transform in the course material

Application	How the Laplace (s-domain) description is used
Describing continuous-time signals and systems	Continuous-time systems and signals are normally described using the Laplace transform; an analogue system is characterized by its transfer function H(s).
Starting point of IIR filter design	IIR coefficients are traditionally calculated by transforming known analogue filter characteristics into equivalent digital filters, using the impulse invariant or the bilinear method.
Converting an analogue filter to a digital one	In Example 5.4 the analogue transfer function H(s) is converted by the bilinear method, at a sampling frequency of 256 Hz, into a digital transfer function H(z) that preserves its amplitude response.
Linking analogue and digital filter types	Analogue filters can be readily transformed into equivalent IIR digital filters; this is not possible with FIR filters, as they have no analogue counterpart.
Interpreting the digital system through the s-plane	Through the mapping $z = e^{sT}$, the frequencies of the s-plane are related to the unit circle of the z-plane (Section 2.6).

The first application is the basic one: the Laplace transform gives the s-domain description H(s) of the analogue filter or system. The second and third applications show why this description matters in digital filtering. The analogue characteristic is known and well established, and the digital filter is obtained by transforming it. The two methods behave differently. The impulse invariant method preserves the impulse response of the analogue filter but not its magnitude–frequency response, and is inappropriate for highpass or bandstop filters because of aliasing. The bilinear method preserves the magnitude response but not the time-domain properties, which is why it was chosen in Example 5.4 to preserve the amplitude response of the analogue function.

2.4 From Continuous Time to Discrete Time: Sampling

To process an analogue signal by digital means it must first be converted into a sequence of numbers of finite precision, which is analogue-to-digital (A/D) conversion. The first step of this conversion is sampling, which converts a continuous-time signal into a discrete-time signal by taking samples at discrete-time instants. In periodic (uniform) sampling, the type used most often in practice, the discrete-time signal x(n) is obtained from $x_a(t)$ by Eq. (2.6), where T is the sampling period (sampling interval). Its reciprocal

$F_s = 1/T$ is the sampling rate or sampling frequency. Periodic sampling links the time variables t and n of the continuous-time and discrete-time signals through Eq. (2.7).

$$x(n) = x_a(nT), \qquad -\infty < n < \infty \tag{2.6}$$

$$t = nT = \frac{n}{F_s}, \qquad F_s = \frac{1}{T} \tag{2.7}$$

As a consequence, a relationship exists between the frequency variables of the two kinds of signals. A discrete-time sinusoid is written x(n) = A cos(Ωn + θ), where n is the sample number, Ω is the frequency in radians per sample and θ is the phase. With the frequency variable F, defined by Ω = 2πF in cycles per sample, it becomes Eq. (2.8). Sampling the analogue sinusoid at the rate F_s and comparing the result with this form shows that F and f are linearly related by $F = f/F_s$, or equivalently by Eq. (2.9).

$$x(n) = A\cos(2\pi F n + \theta), \qquad F = \frac{f}{F_s} \tag{2.8}$$

$$\Omega = 2\pi F = \omega T \tag{2.9}$$

Eq. (2.8) can be used to find the frequency f in hertz only if the sampling frequency F_s is known. The range of the frequency variable is also different. For continuous-time sinusoids $-\infty < f < \infty$, as noted in Section 2.1, whereas for discrete-time sinusoids the frequencies in the range of Eq. (2.10) are unique. In terms of the analogue frequency this means that, with sampling at the rate $F_s = 1/T$, the frequency f must fall in the corresponding range of Eq. (2.10). The highest frequencies that can be represented are therefore $f_{max} = F_s/2 = 1/(2T)$ and $\omega_{max} = \pi/T$.

$$-\frac{1}{2} \le F \le \frac{1}{2}, \qquad -\pi \le \Omega \le \pi$$

$$-\frac{F_s}{2} \le f \le \frac{F_s}{2}, \qquad -\frac{\pi}{T} \le \omega \le \frac{\pi}{T} \tag{2.10}$$

Table 6 Relations among the frequency variables

Item	Continuous-time signals	Discrete-time signals
Frequency in radians	ω (rad/s)	Ω (rad/sample)
Frequency in cycles	f (Hz)	F (cycles/sample)
Relation	ω = 2πf	Ω = 2πF
Link through sampling	$F = f/F_s$	Ω = ωT
Range of frequency	$-\infty < f < \infty$	$-1/2 \le F \le 1/2$, $-\pi \le \Omega \le \pi$

Discrete-time sinusoids differ from continuous-time ones in three further respects. First, a discrete-time sinusoid is periodic only if its frequency F is a rational number, F = k/N, and its fundamental period is N after cancelling common factors; a small change in

frequency can therefore produce a large change in the period (for example F = 31/60 gives N = 60, whereas F = 30/60 gives N = 2). Second, discrete-time sinusoids whose frequencies are separated by an integer multiple of 2π are identical, because $\cos[(\Omega_0 + 2\pi)n + \theta] = \cos(\Omega_0 n + \theta)$. A sinusoid with $|\Omega| > \pi$ (or $|F| > 1/2$) is therefore identical to one with a frequency inside the unique range and is called an alias of it. Third, the highest rate of oscillation of a discrete-time sinusoid is reached at $\Omega = \pi$ ($F = 1/2$); for $\Omega_0 = 0, \pi/8, \pi/4, \pi/2$ and π the sequences have the periods ∞, 16, 8, 4 and 2, so the period decreases as the frequency increases.

To avoid the problem of aliasing, the sampling rate is selected according to the sampling theorem, Eq. (2.11), where f_{max} is the largest frequency component in the analogue signal. If the highest frequency contained in $x_a(t)$ is $f_{max} = B$ and the signal is sampled at a rate $F_s > 2B$, then $x_a(t)$ can be recovered exactly from its samples. The rate $2f_{max}$ is called the Nyquist rate. For example, the major frequency components of speech fall below 3000 Hz, so $f_{max} = 3000$ Hz for the class of speech signals, and the sampling rate must exceed 6000 Hz.

$$F_s > 2f_{max} \tag{2.11}$$

2.5 The Z-Transform

The Z-transform of a sequence x(n) that is valid for all n is defined by Eq. (2.12), where z is a complex variable. In causal systems x(n) may be nonzero only for $0 \le n < \infty$, and the definition reduces to the one-sided z-transform of Eq. (2.13).

$$X(z) = \sum_{n=-\infty}^{\infty} x(n)\, z^{-n} \tag{2.12}$$

$$X(z) = \sum_{n=0}^{\infty} x(n)\, z^{-n} \tag{2.13}$$

The region where the z-transform converges, so that the values of X(z) are finite, is called the region of convergence (ROC). For the causal infinite-duration sequence of Example 4.1(d), whose samples are 1 for $n \ge 0$, the transform is a geometric series in z^{-1} with the common ratio z^{-1}. It converges when $|z| > 1$, and X(z) can then be written in the closed form of Eq. (2.14). The ROC is the exterior of the circle of unit radius centred at the origin, bounded by the circle $|z| = 1$, the radius of the pole of X(z). For z = 2 the series adds up to 2, while for z = 1/2 it diverges.

$$X(z) = 1 + z^{-1} + z^{-2} + \cdots = \frac{1}{1 - z^{-1}}, \qquad |z| > 1 \tag{2.14}$$

The values of z for which X(z) becomes infinite are the poles of X(z), and the values for which X(z) = 0 are its zeros. From the examples of the reference three conclusions are drawn: for causal sequences of finite duration the z-transform converges everywhere except at z = 0; for causal infinite-duration sequences it converges everywhere outside a

circle bounded by the radius of the pole with the largest radius; and for stable causal systems the ROC always encloses the circle of unit radius, which is important for the systems to have a frequency response.

Properties and system description

Several properties of the z-transform are used in DSP. Linearity means that the transform of a linear combination of sequences is the same linear combination of their transforms. The delay property, Eq. (2.15), says that delaying a sequence by m samples multiplies its transform by z^{-m}; it is widely used to convert the transfer function of a discrete-time system into a time-domain difference equation and vice versa. The convolution property, Eq. (2.16), says that the convolution sum relating the output y(n) of an LTI system to its input x(n) and impulse response h(k) becomes a multiplicative process in the z-domain, where X(z), H(z) and Y(z) are the transforms of x(n), h(k) and y(n). H(z) is often called the system transfer function. A fourth property, differentiation, gives the transform of n x(n) by differentiating X(z) and is useful for inverse transforms with multiple-order poles.

$$x(n-m) \longleftrightarrow z^{-m}X(z) \tag{2.15}$$

$$y(n)=\sum_{k=0}^{\infty} h(k)\,x(n-k) \longleftrightarrow Y(z)=H(z)\,X(z) \tag{2.16}$$

The difference equation of a discrete-time system specifies the operations the system performs, in the time domain, on the input data to produce the output. For most practical cases it is written as Eq. (2.17), and using the delay property the z-domain transfer function is obtained as Eq. (2.18).

$$y(n)=\sum_{k=0}^{N} a_k\,x(n-k)-\sum_{k=1}^{M} b_k\,y(n-k) \tag{2.17}$$

$$H(z)=\frac{Y(z)}{X(z)}=\frac{\sum_{k=0}^{N} a_k z^{-k}}{1+\sum_{k=1}^{M} b_k z^{-k}} \tag{2.18}$$

If all the denominator coefficients b_k are zero, the system output depends only on the present and past inputs and the system is an FIR system; if at least one of them is nonzero it is an IIR system. If H(z) has the poles $p_1, \ldots, p_N$ and the zeros $z_1, \ldots, z_N$, it can be written in the factored form of Eq. (2.19), where K is the gain factor. Poles and zeros may be real or complex, and when complex they occur in conjugate pairs so that the coefficients a_k and b_k are real. The information in the z-transform is conveniently shown in a pole-zero diagram, in which X marks a pole and o marks a zero, together with the

unit circle |z| = 1, which plays an important role in the analysis and design of discrete-time systems.

$$H(z) = \frac{K(z - z_1)(z - z_2)\cdots(z - z_N)}{(z - p_1)(z - p_2)\cdots(z - p_N)} \tag{2.19}$$

Stability. A useful stability criterion for LTI systems is that every bounded input produces a bounded output (BIBO stability). A system is BIBO stable if and only if Eq. (2.20) holds, where h(k) is the impulse response. A finite-length impulse response satisfies it automatically, so stability considerations apply only to systems with an impulse response of infinite duration. For the output to be bounded all the poles must lie inside the unit circle. A pole outside the unit circle makes the system unstable, and a pole on the unit circle is regarded as unstable or potentially unstable, except when it coincides with a zero on the unit circle. For an unstable system the impulse response increases indefinitely with time.

$$\sum_{k=0}^{\infty} |h(k)| < \infty \tag{2.20}$$

The behaviour of a simple system with the z-transform $10z^{-1}/(1 - \alpha z^{-1})$ in the reference illustrates this: for $\alpha = 0.5$ the system is stable and the impulse response decays rapidly; for $\alpha = 0.99$ it is marginally stable; for $\alpha = 1$ it is potentially unstable; and for $\alpha = 1.5$ it is unstable and the impulse response increases rapidly.

Impulse response. The impulse response of a discrete-time system is the inverse z-transform of its transfer function, $h(k) = Z^{-1}[H(z)]$, k = 0, 1, …. It is needed to implement an FIR system and for the stability analysis of an IIR system, and it can be used to evaluate the frequency response. It can be obtained from the power series of H(z), whose coefficients give h(k) directly, or by applying a unit impulse to the system. In Example 4.11, $H(z) = (1 - z^{-1})/(1 + 0.5z^{-1})$ gives h(0) = 1, h(1) = −1.5, h(2) = 0.75 and h(3) = −0.375 by either method.

2.6 Relationship Between the Laplace Transform and the Z-Transform

The relationship is obtained by letting $z = e^{sT}$, where s is the complex Laplace variable of Eq. (2.5) and T is the sampling interval, $T = 1/F_s$. Then:

$$z = e^{sT} = e^{(d + j\omega)T} = e^{dT} e^{j\omega T} \tag{2.21}$$

Thus the magnitude and the angle of z are given by Eq. (2.22), where ω_s (rad/s) is the sampling frequency. The angle of z, ωT, is the same quantity as the discrete-time frequency Ω = ωT of Eq. (2.9): a point of the s-plane with the angular frequency ω is mapped to the point of the z-plane whose angle is the discrete-time frequency of the sampled signal.

$$|z| = e^{dT}, \qquad \angle z = \omega T = \frac{2\pi f}{F_s} = \frac{2\pi\omega}{\omega_s} \tag{2.22}$$

As ω varies from −∞ to ∞, the s-plane is mapped onto the z-plane as shown in Fig. 5. The mapping has the following properties:

- The entire jω axis of the s-plane is mapped onto the unit circle of the z-plane.
- The left-hand half of the s-plane is mapped to the inside of the unit circle.
- The right-hand half of the s-plane is mapped to the outside of the unit circle.

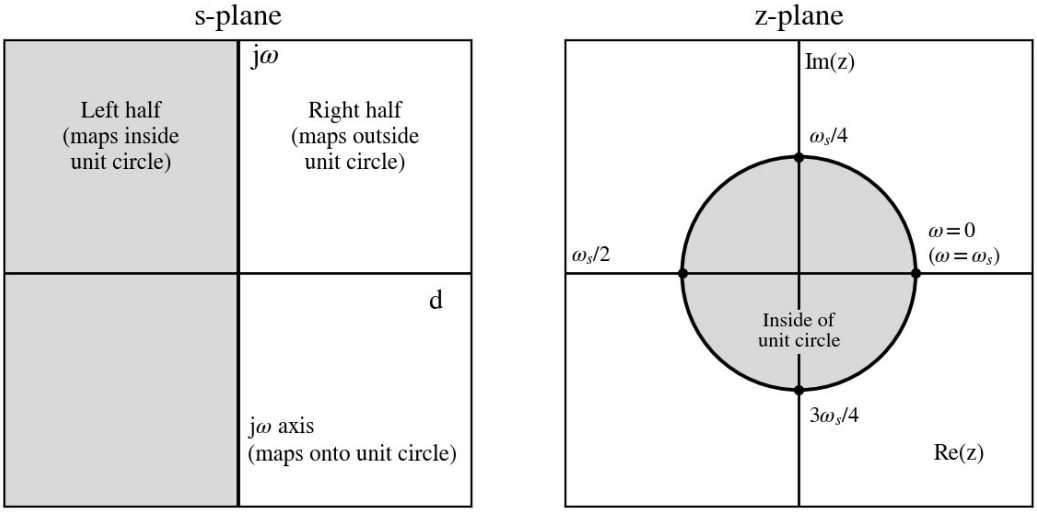


Fig. 5 Mapping of the s-plane to the z-plane. The left-hand side of the s-plane maps to the interior of the unit circle, the right-hand side maps to the exterior, and the jω axis maps onto the unit circle.

In terms of frequency response the jω axis is the most important one. On it d = 0, and the frequency points of the s-plane are related to the points of the unit circle by Eq. (2.23).

$$z = e^{j\omega T} \tag{2.23}$$

Table 7 shows how specific frequencies are mapped from the s-plane to the z-plane, with the angle computed from Eq. (2.22), $\omega T = 2\pi\omega/\omega_s$.

Table 7 Mapping of frequencies from the s-plane to the z-plane

s-plane: ω (rad/s)	**z-plane: ωT (rad)**
0	0
$\omega_s/4$	π/2
$\omega_s/2$	π
$3\omega_s/4$	1.5π
ω_s	2π
$1.25\omega_s$	π/2 (+2π)
$1.5\omega_s$	π (+2π)
$1.75\omega_s$	1.5π (+2π)
$2\omega_s$	2π (+2π)

The table shows that the mapping is not unique. For example, the two frequencies $\omega = \omega_s$ and $\omega = 2\omega_s$ in the s-plane map to the same point on the unit circle, and $1.25\omega_s$ falls on the same point as $\omega_s/4$. This agrees with the sampling results of Section 2.4: two analogue frequencies that differ by $\omega_s = 2\pi/T$ give discrete-time frequencies $\Omega = \omega T$ that differ by 2π, and discrete-time sinusoids whose frequencies are separated by an integer multiple of 2π are identical. Different analogue frequencies can therefore correspond to the same point of the z-plane, which is consistent with the remark in Section 1.7.2 that aliasing is inherent in the impulse invariant method. The unique discrete-time frequencies, $-\pi \le \Omega \le \pi$, cover the unit circle once.

The relationship also connects with stability. For a discrete-time system to be stable, all the poles of its transfer function must lie inside the unit circle (Section 2.5), and the left-hand half of the s-plane is mapped to the inside of the unit circle. Combining the two statements shows that the left-hand half of the s-plane corresponds to the region in which the poles of a stable discrete-time system lie. In the same way the ROC of a stable causal system encloses the unit circle, which is the image of the $j\omega$ axis, so that the system has a frequency response.

2.7 Numerical Illustration of the Mapping

The mapping can be illustrated with the sampling frequency of Example 5.3, $F_s = 128$ Hz. A frequency f corresponds to $F = f/F_s$ and to the point on the unit circle at the angle $\omega T = \Omega = 2\pi F$, from Eqs. (2.8), (2.9) and (2.22) (Table 8).

Table 8 Angle on the unit circle for $F_s = 128$ Hz

f (Hz)	$F = f/F_s$	$\Omega = 2\pi F$ (rad)	Remark
0	0	0	Point z = 1
10	0.078	0.49 ($\approx 0.16\pi$)	Passband edge of Example 5.3
20	0.156	0.98 ($\approx 0.31\pi$)	Stopband edge of Example 5.3
32	0.25	$\pi/2$	$F_s/4$
64	0.5	π	Highest rate of oscillation ($F_s/2$)
128	1	2π	Same point as 0 Hz
160	1.25	$2.5\pi \equiv \pi/2$	Alias of 32 Hz

The table confirms the properties found above. The frequencies between 0 and $F_s/2$ occupy the upper half of the unit circle, the highest unique frequency $F_s/2$ sits at $z = -1$, and a frequency larger than $F_s/2$ lands on a point already used by a lower frequency.

2.8 Summary of the Relationship

Table 9 Laplace transform versus z-transform

Item	Laplace transform	Z-transform
Systems and signals described	Continuous-time	Discrete-time
Complex variable	$s = d + j\omega$	$z = e^{sT}$
Transfer function	H(s), e.g. the analogue filter of Example 5.4	H(z), e.g. Eqs. (5.4a) and (5.4b)
Plane and frequency axis	s-plane; the jω axis	z-plane; the unit circle
Left-hand side	Left-hand half of the s-plane	Maps to the inside of the unit circle
Right-hand side	Right-hand half of the s-plane	Maps to the outside of the unit circle
Use in filter design	Analogue filter characteristics	Digital IIR and FIR filters

In summary, the Laplace transform describes continuous-time systems and signals in the s-plane, the z-transform plays the same role for discrete-time systems in the z-plane, and the substitution $z = e^{sT}$ links the two: the jω axis becomes the unit circle, the left-hand half-plane becomes its inside and the right-hand half-plane its outside. Through sampling, the angle of z is the discrete-time frequency $\Omega = \omega T$. This link is what allows known analogue filter characteristics to be transformed into equivalent IIR digital filters.

References

[1] Samir Kamal Younis, *Digital Signal Processing (ELC 422), Fifth Year Communications (Parts Selected)*, Modern Academy in Maadi for Engineering & Technology.

[2] Samir Kamal and Sara Hassan, *DSP lecture notes*, Modern Academy for Engineering and Technology: Lectures 3 and 4 (Chapter 1), Lectures 9, 10-2 and 11 (Chapter 4).